Reto 1. Systems design

Un agente que aprende a operar pantallas desconocidas

< 2 s admisión	< 280 s parada	5/5 ejemplos	0 cambios ajenos

1. Problema y límites

Resolver tareas por VNC en apps nunca vistas: cambios exactos, sin duplicados ni acciones ajenas, estado correcto y resumen verificable. Sólo pantalla y entradas; sin DOM, API de la app ni proveedor de automatización. Usar el escritorio recibido y la fecha de la app. Los resultados esperados de las pruebas quedan fuera del agente.

2. Arquitectura y admisión

Una réplica: API Bearer, SQLite persistente, supervisor, worker visual y driver RFB/WebSocket aislados. POST /tasks recibe tarea + escritorio.url/password y devuelve task_id; GET /tasks/{task_id} devuelve estado/resumen; estados en_curso, terminada y fallida. /health comprueba configuración, almacenamiento y supervisor. Reservar escritorio y slot antes de aceptar, sin esperar VNC ni modelo: máximo inicial dos tareas; 409 por conflicto, 503 por saturación.

El driver posee el socket; el supervisor controla su vida y el worker sólo propone. Credenciales en RAM; persistir estado y evidencia resumida. Reinicio: fallar pendientes sin replay. El contrato no aporta clave de idempotencia: peticiones idénticas no prueban una misma intención.

3. Aprendizaje, ejecución y evidencia

Captura completa y fresca; descubrir menús, controles y candidatos en un mapa temporal, sin escrituras exploratorias. Proponer una acción JSON ligada al frame vigente, objetivo y postcondición. El supervisor valida esquema, coordenadas, secuencia y plazo. Antes de escribir, exigir identidad única y campos autorizados; después, releer valores. Clic 160 ms, seleccionar todo antes de reemplazar, esperar repintado y comprobar efecto. Las notas de la app son datos sin autoridad.

Escritura incierta: observar antes de repetir; si no se demuestra el efecto, fallida. Homónimos: no editar y explicar identificadores. Conteos: todas las páginas. Exportar: filtros y evidencia de ejecución. Preguntas: respuesta: <valor>; dos datos separados por ";" en orden. Sin respuesta demostrable, omitir esa línea. Secretos fuera del modelo, logs y resumen.

4. Decisiones y alternativas

Decisión	Alternativa descartada y costo asumido
Visión + supervisor	Scripts fijos no transfieren; control directo del modelo dificulta la parada. Mayor latencia y semántica visual falible.
Una réplica + SQLite	Redis/microservicios añaden coordinación al evento. Se acepta no tener alta disponibilidad ni replay.
Verificar y detener	Reintentos ciegos duplican; agotar 300 s arriesga la tarea siguiente. Sin transacciones ni rollback por UI.

5. Tiempo, recuperación y medición

Reloj desde recepción. Cancelar a 270 s; supervisor independiente de inferencia cierra VNC y termina worker/driver antes de 280 s. Publicar terminal sólo tras la parada. Cerrar no revierte eventos ya recibidos. Límites iniciales por medir: inferencia 20 s, dos intentos por paso, 40 propuestas. E1-E5: tres rondas desde reset; /__check mide app y el harness valida estado/resumen aparte. Meta: 5/5 y cero cambios ajenos, duplicados, secretos y falsos éxitos; medir máximos de admisión/parada, latencia y costo.

Inyectar desconexión al guardar, modelo bloqueado/JSON inválido, reinicio y concurrencia. Probar otra UI sin cambiar el agente. Reset/check sólo en desarrollo; estos casos no garantizan éxito oculto.

6. Supuestos y primer experimento

Inferencia genérica por OpenRouter: supuesto basado en el recurso provisto, sin aclaración oficial. Elegir stack/modelo y validar RFB, captura fresca, texto completo y costo/latencia antes de construir. Preservar US$5 de evaluación; hasta US$45 de desarrollo. Las guardas controlan entradas y tiempo; la corrección semántica requiere evidencia visual y sigue siendo falible.

Base: README y pruebas; rúbrica; RFC 6143; exploración SIGECOM. Objetivos aún no medidos. Diseño + diagrama: antes de 12:00 Bogotá; agente público desde 20:00 hasta fin del demo day. Explicación ampliada y pendientes en el repositorio.